# Brief Overview

HaifaU **DLC** (**D**eep **L**earning **C**luster) consists of 6 NVIDIA DGX A100 servers with 8 NVIDIA A100 40GB GPUs each, fast 200Gb/s HDR InfiniBand interconnect and supporting infrastructure. Compute jobs scheduling is managed by SLURM. Jobs are running on the optimized Nvidia containers layer.

## Cluster nodes

| Host Name | Role | IP |
|---|---|---|
| login01.dlc.cs.haifa.ac.il | Login node | 132.75.245.170 |
| dgx01 .. dgx06 | Compute nodes | |

# Submitting and Controlling Jobs

Jobs can be submitted either with `sbatch` submission scripts that also define resource requirements and what the system should do once the job runs, or interactively using `srun`, which can be very useful for testing and debugging.

### Pulling NGC container

Use `/dlc/sw/bin/pull_container_to_file.sh` to pull container image in preparation for running a job in the cluster, providing a tag from NGC containers catalog.
Be patient, the script may take a while to run.

Example:

```
test1@login01:~$ /dlc/sw/bin/pull_container_to_file.sh
nvcr.io/nvidia/pytorch:21.04-py3
Going to pull nvcr.io/nvidia/pytorch:21.04-py3 from NGC
pyxis: importing docker image ...
pyxis: creating container filesystem ...
pyxis: starting container ...
Container image saved to /users/testers/test1 as pytorch:21.04-py3.sqsh
Use srun --container-image ~/pytorch:21.04-py3.sqsh <parameters> to submit a job
using this container
Add --container-save ~/pytorch:21.04-py3.sqsh parameter to persist changes
```

## Submitting Interactive Job

Use srun to submit interactive jobs.

### Commonly used SRUN options

| | |
|---|---|
| `-p partitionName` | submit a job to partition partitionName. By default “`batch`” partition is selected. |
| `-o output.log` | Append job's output to output.log instead of `slurm-%j.out` in the current directory |
| `-e error.log` | Append job's STDERR to error.log instead of job output file (see -o above) |
| `-G N`<br>`--gpus N` | Allocate N GPUs *per node* for the job |
| `-n N`<br>`--ntasks N` | Set number of processors (cores) to N(default=1), the cores will be allocated to cores chosen by SLURM |
| `-J jobName`<br>`--job-name jobName` | Set the job name to be shown in the queue. |
| `--pty` | Allocate PTY for the job (run task zero in pseudo terminal) |
| `--container-image=[USER@][REGISTRY#]IMAGE[:TAG]|PATH` | The image to use for the container filesystem. Can be either a docker image given as an enroot URI, or a path to a squashfs file on the remote host filesystem. |
| `--container-save=PATH` | Save the container state to a squashfs file |
| `--container-mounts=SRC:DST[:FLAGS][,SRC:DST...]` | bind mount[s] inside the container. Mount flags are separated with "+", e.g. "ro+rprivate" |
| `--container-entrypoint`<br>`--no-container-entrypoint` | Control execution of the entrypoint from the container |

## Examples

Interactive single-node job

```
# Run a container from image imported from NGC

$ srun  --container-image=/users/testers/test1/pytorch:21.04-py3.sqsh
--container-save=/users/testers/test1/pytorch:21.04-py3.sqsh apt install -y vmtouch

# We can reuse the existing container and execute our new binary on the dataset
$ srun --container-image=/users/testers/test1/pytorch:21.04-py3.sqsh
--container-mounts=/users/testers/test1/datasets:/datasets vmtouch /datasets
```

```
# Now we can start an interactive session inside the container, for
development/debugging:
$ srun --gpus=1 --container-image=/users/testers/test1/pytorch:21.04-py3.sqsh
--container-mounts=/users/testers/test1/datasets:/datasets --pty /bin/bash
...
root@dgx01:/workspace# nvidia-smi
...
```

# Submitting Batch Job

Use `sbatch` to submit single- or multi-node batch jobs

## Commonly used SBATCH options

| | |
|---|---|
| `-p partitionName` | submit a job to partition partitionName. By default "`batch`" partition is selected. |
| `-o output.log` | Append job's output to *`output.log`* instead of `slurm-%j.out` in the current directory |
| `-e error.log` | Append job's STDERR to *`error.log`* instead of job output file (see -o above) |
| `-G N`<br>`--gpus N` | Allocate N GPUs *per node* for the job |
| `-n N`<br>`--ntasks N` | Set number of processors (cores) to N(default=1), the cores will be allocated to cores chosen by SLURM |
| `-N N`<br>`--nodes N` | Set the number of nodes that will be part of a job.<br>`--ntasks-per-node` processes started on each job node.<br>One process per node will be started if `--ntasks-per-node` is not specified |
| `--ntasks-per-node N` | How many tasks per allocated node to start (see -N above) |
| `--cpus-per-task N` | Needed for multithreaded (e.g. OpenMP) jobs.<br>The option tells SLURM to allocate N cores per task allocated; typically N should be equal to the number of threads the program spawns, e.g. it should be set to the same number as `OMP_NUM_THREADS` |
| `-J jobName`<br>`--job-name jobName` | Set the job name to be shown in the queue. |
| `-D directory`<br>`--chdir=directory` | set working directory for batch script |
| `-t minutes`<br>`--time=minutes` | time limit |

## Examples

Single-node two-GPU job

```
$ cat job.sh
#!/bin/bash
#SBATCH -J tensorflow-benchmark
#SBATCH -o %x.%j.out
#SBATCH -e %x.%j.err
#SBATCH -D /users/testers/test1
#SBATCH --time=01:00:00
#SBATCH -G 2
#SBATCH --get-user-env
#SBATCH --nodes 1
git clone https://github.com/tensorflow/benchmarks/

srun --ntasks=2 --mpi=mpi2
--container-image="nvcr.io#nvidia/tensorflow:20.12-tf2-py3" \
 /bin/bash -c "python
/root/benchmarks/scripts/tf_cnn_benchmarks/tf_cnn_benchmarks.py --num_gpus=2
--batch_size=200 --model=resnet50 --variable_update=horovod --use_fp16"

$ sbatch job.sh
```

Multi-node job

```
$ cat job.sh
#!/bin/bash

# You can start a container on each node from a shared squashfs file
srun --nodes=4 --ntasks-per-node=16 --mpi=pmix \
     --container-image=/users/testers/test1/tensorflow.sqsh \
     --container-mounts=/users/testers/test1/datasets:/datasets \
     python train.py /datasets/cats
# The container rootfs get automatically cleaned up at the end of the script
echo "Training complete!"

# Submitting the job is done through the command-line too
$ sbatch --nodes=4 --output=tensorflow_training.log job.sh
```

# Displaying Cluster Info

Use `sinfo` to view information about Slurm nodes and partitions.

## Examples

```
$ sinfo -l
Sun Sep 1 11:27:59 2021
PARTITION AVAIL   TIMELIMIT  JOB_SIZE ROOT OVERSUBS  GROUPS  NODES   STATE NODELIST
batch*       up   infinite 1-infinite   no    NO     all     1       mixed dgx01
batch*       up   infinite 1-infinite   no    NO     all     1   allocated dgx02
batch*       up   infinite 1-infinite   no    NO     all     4        idle dgx[03-06]
```

# Displaying Queued Jobs

Use `squeue` to show queued jobs in different states (`PENDING, SUSPENDED`, etc..) or recently finished jobs (`COMPLETED, CANCELED`, etc...)

### Commonly used SQUEUE options

| | |
|---|---|
| `-p partitionName` | Show job states in partition *`partitionName`* |
| `-t statesList`<br>`--states=statesList` | Specify the states of jobs to view. *`statesList`* should be either a comma-separated list of states (`RUNNING, PENDING, SUSPENDED`, etc.. ) or `all` to show all states.<br><br>If not specified - queue shows only jobs in `PENDING, RUNNING` and `COMPLETING` states |
| `-l` | Report more of the available information |

## Examples

```
$ squeue -l
Sun Sep 1 11:31:22 2021
JOBID PARTITION  NAME     USER   STATE        TIME TIME_LIMI  NODES NODELIST(REASON)
  377  batch     resnet1   john   COMPLETI     0:37 UNLIMITED      1 dgx01
  378  batch    bash      test1  RUNNING      0:27 UNLIMITED      1 dgx02
```

# Killing Jobs

Use `scancel *jobID*` to kill a running or pending job.
To kill all jobs for user userName run `scancel -u *userName*`